

IMPORTS ET PRÉSENTATION

DESCRIPTION : math fournit les fonctions mathématiques pour réels (float/int).
cmath fournit les mêmes fonctions pour nombres complexes.
Les deux sont des modules de la bibliothèque standard Python,
implémentés en C, très rapides pour les scalaires.

POURQUOI : Calculs mathématiques sans dépendances externes (pas de numpy).

CONTEXTE : Scripts légers, calculs scalaires, environnements sans numpy.

IMPORTS :

import math	Module pour réels
import cmath	Module pour complexes
from math import pi, e, sqrt, log	Import sélectif
from cmath import phase, polar, rect	Import sélectif

DIFFÉRENCES CLÉS math vs cmath :

math.sqrt(-1)	→ ValueError (domaine invalide)
cmath.sqrt(-1)	→ 1j (complexe)
math.log(-1)	→ ValueError
cmath.log(-1)	→ 3.141592653589793j
type(math.sqrt(4))	→ float (toujours)
type(cmath.sqrt(4))	→ complex (toujours)
math.isnan(float("nan"))	→ True
math.isinf(float("inf"))	→ True

COMPATIBILITÉ AVEC NUMPY :

```
# math et cmath ne fonctionnent PAS sur des tableaux numpy
math.sqrt(np.array([4])) → TypeError (utiliser np.sqrt)
# Mais fonctionnent sur des scalaires numpy
math.sqrt(np.float64(4)) → 2.0 (OK)
# Pour les tableaux, toujours utiliser numpy
```

[math] Constantes

DESCRIPTION : Constantes mathématiques prédéfinies de haute précision.

POURQUOI : Éviter de taper des approximations, garantir la précision maximale.

CONTEXTE : Calculs trigonométriques, probabilités, physique.

CONSTANTES MATHÉMATIQUES :

math.pi	$\pi = 3.141592653589793$
math.e	$e = 2.718281828459045$
math.tau	$\tau = 2\pi = 6.283185307179586$
math.inf	$+\infty$ (float("inf"))
math.nan	NaN (float("nan"))
math.sqrt(2)	$\sqrt{2} \approx 1.4142135623730951$
# Pas de constante ϕ (nombre d'or), mais facile à calculer :	
phi = (1 + math.sqrt(5)) / 2	$\phi \approx 1.618033988749895$

EXEMPLES :

```
# Conversion degrés ↔ radians
rad = 180 * math.pi / 180 →  $\pi$ 
rad = math.radians(180) →  $\pi$  (méthode recommandée)
deg = math.degrees(math.pi) → 180.0
```

[math] Fonctions d'arrondi et entières

DESCRIPTION : Arrondi vers entiers, partie entière, modulo et division entière.

POURQUOI : Travailler avec des entiers à partir de flottants.

CONTEXTE : Calculs financiers, indices, discrétisation.

ARRONDI :

math.floor(x)	$\lfloor x \rfloor$ – entier $\leq x$ (arrondi vers $-\infty$)
math.floor(3.7)	→ 3
math.floor(-3.7)	→ -4
math.ceil(x)	$\lceil x \rceil$ – entier $\geq x$ (arrondi vers $+\infty$)
math.ceil(3.2)	→ 4
math.ceil(-3.2)	→ -3

```

math.trunc(x)
math.trunc(3.7)
math.trunc(-3.7)
round(x)
round(3.5)
round(2.5)
round(3.14159, 2)
math.floor(x) vs int(x)
int(-3.7)
math.floor(-3.7)

```

Troncature vers 0 (partie entière)
→ 3
→ -3
Arrondi standard Python (built-in)
→ 4 (banker's rounding)
→ 2 (arrondi vers pair)
→ 3.14
Différent pour négatifs !
→ -3 (troncature)
→ -4 (vers $-\infty$)

ENTIERS :

```

math.factorial(n)
math.factorial(0)
math.factorial(10)
math.factorial(100)
math.comb(n, k)
math.comb(10, 3)
math.comb(10, 0)
math.comb(10, 10)
math.perm(n, k=None)
math.perm(5, 3)
math.perm(5)
math.gcd(a, b)
math.gcd(12, 8)
math.gcd(0, 5)
math.gcd(12, 8, 6)
math.lcm(a, b)
math.lcm(4, 6)
math.lcm(4, 6, 9)
math.isqrt(n)
math.isqrt(16)
math.isqrt(17)
math.isqrt(10**20)

```

$n!$ – entier exact
→ 1
→ 3628800
entier Python exact (grand)
 $C(n,k) = n!/(k!(n-k)!)$ – combinaisons
→ 120
→ 1
→ 1
 $P(n,k) = n!/(n-k)!$ – permutations
→ 60
→ 120 (= 5!)
PGCD (plus grand commun diviseur)
→ 4
→ 5
PGCD de plusieurs entiers (Python 3.9+)
PPCM (plus petit commun multiple)
→ 12
PPCM de plusieurs (Python 3.9+)
 $\lfloor \sqrt{n} \rfloor$ – racine carrée entière exacte
→ 4
→ 4
→ 10000000000 (grand entier OK)

ARITHMÉTIQUE FLOTTANTE :

```

math.fmod(x, y)
math.fmod(5.5, 2.2)
math.fmod(-5.5, 2.2)
5.5 % 2.2
math.remainder(x, y)
math.remainder(5.5, 2.2)
math.divmod(x, y)
# Utiliser divmod() built-in
divmod(7, 2)
math.modf(x)
math.modf(3.7)
math.modf(-3.7)
frac, entier = math.modf(3.7)
math.frexp(x)
math.frexp(8.0)
math.frexp(1.5)
m, e = math.frexp(x)
math.ldexp(m, e)
math.ldexp(0.5, 4)

```

Modulo flottant (signe de x)
→ 1.1
→ -1.1 (différent de %)
→ 1.1000...6 (artefact flottant)
Reste IEEE 754 (arrondi au plus proche)
→ -1.1 ($|reste| \leq y/2$)
(quotient, reste) – non disponible
→ (3, 1)
→ (partie_fractionnaire, partie_entière)
→ (0.7000000000000002, 3.0)
→ (-0.7000000000000002, -3.0)
→ (mantis, exposant) où $x = m * 2^e$
→ (0.5, 4) ($0.5 * 2^4 = 8.0$)
→ (0.75, 1)
 $m * 2^e$ (inverse de frexp)
→ 8.0

EXEMPLES :

```

# Division euclidienne précise
q = math.floor(7.5 / 2.5)
r = math.fmod(7.5, 2.5)
# PGCD et PPCM
g = math.gcd(48, 36)
l = math.lcm(4, 6)

```

→ 3
→ 0.0
→ 12
→ 12

[math] Puissances, racines et logarithmes

DESCRIPTION : Fonctions de puissance, racines et logarithmes pour réels.
POURQUOI : Calculs courants en science, finance et ingénierie.
CONTEXTE : Transformations de données, croissance exponentielle, entropie.

PUISSANCES ET RACINES :

```

math.sqrt(x)
math.sqrt(4)
math.sqrt(2)

```

$\sqrt{x}$ – racine carrée
→ 2.0
→ 1.4142135623730951

<code>math.sqrt(0)</code>	→ 0.0
<code>math.sqrt(-1)</code>	→ ValueError (utiliser cmath)
<code>math.pow(x, y)</code>	x^y – retourne toujours float
<code>math.pow(2, 10)</code>	→ 1024.0
<code>math.pow(2, -1)</code>	→ 0.5
<code>math.pow(2, 0.5)</code>	→ $\sqrt{2}$
<code>math.pow(0, 0)</code>	→ 1.0
<code>x ** y</code>	Opérateur puissance Python (peut retourner int)
<code>pow(x, y)</code>	Identique à <code>**</code> pour flottants
<code>pow(x, y, mod)</code>	Puissance modulaire (entiers uniquement)
<code>pow(2, 10, 1000)</code>	→ 24 ($2^{10} \bmod 1000$)
<code>math.cbrt(x)</code>	$\sqrt[3]{x}$ – racine cubique (Python 3.11+)
<code>math.cbrt(8)</code>	→ 2.0
<code>math.cbrt(-8)</code>	→ -2.0 (supporte les négatifs)
<code># Racine n-ième</code>	
<code>x ** (1/n)</code>	Racine n-ième (attention aux négatifs)
<code>math.pow(x, 1/n)</code>	Identique
<code>abs(x) ** (1/n) * math.copysign(1, x)</code>	Racine n-ième signée

EXPONENTIELLES :

<code>math.exp(x)</code>	e^x
<code>math.exp(0)</code>	→ 1.0
<code>math.exp(1)</code>	→ 2.718281828459045 (= e)
<code>math.exp(-math.inf)</code>	→ 0.0
<code>math.exp(math.inf)</code>	→ inf
<code>math.expm1(x)</code>	$e^x - 1$ (précis pour $x \approx 0$)
<code>math.expm1(1e-10)</code>	→ 1.000000000005e-10 (précis)
<code>math.exp(1e-10) - 1</code>	→ 1.0000000000500001e-10 (moins précis)
<code>math.exp2(x)</code>	2^x (Python 3.11+)
<code>math.exp2(10)</code>	→ 1024.0

LOGARITHMES :

<code>math.log(x)</code>	$\ln(x)$ – logarithme naturel (base e)
<code>math.log(math.e)</code>	→ 1.0
<code>math.log(1)</code>	→ 0.0
<code>math.log(0)</code>	→ ValueError (-inf serait correct)
<code>math.log(-1)</code>	→ ValueError (utiliser cmath)
<code>math.log(x, base)</code>	<code>log_base(x)</code>
<code>math.log(8, 2)</code>	→ 3.0
<code>math.log(1000, 10)</code>	→ 2.9999999999999996 (imprécis)
<code>math.log10(x)</code>	<code>log_10(x)</code> – précis (méthode dédiée)
<code>math.log10(1000)</code>	→ 3.0 (exact)
<code>math.log10(1)</code>	→ 0.0
<code>math.log2(x)</code>	<code>log_2(x)</code> – précis (méthode dédiée)
<code>math.log2(8)</code>	→ 3.0 (exact)
<code>math.log2(1)</code>	→ 0.0
<code>math.log1p(x)</code>	$\ln(1+x)$ – précis pour $x \approx 0$
<code>math.log1p(1e-10)</code>	→ 9.999999999500001e-11 (précis)
<code>math.log(1 + 1e-10)</code>	→ 1.0000000000000002e-10 (moins précis)
<code># Règle : toujours préférer log10 et log2 aux versions log(x, base)</code>	
<code># Règle : toujours préférer expm1/log1p quand l'argument est proche de 0/1</code>	

EXEMPLES :

```
# Décroissance exponentielle
def decay(t, t_half): return math.exp(-math.log(2) * t / t_half)
# Entropie de Shannon (bits)
def entropy_bit(p): return -math.log2(p) if p > 0 else 0
# Nombre de chiffres d'un entier
n_digits = math.floor(math.log10(abs(n))) + 1
```

[math] Trigonométrie

DESCRIPTION : Fonctions trigonométriques directes, inverses et hyperboliques.
POURQUOI : Géométrie, signaux, rotations, physique ondulatoire.
CONTEXTE : Calculs géométriques, traitement du signal, robotique.

CONVERSION DEGRÉS / RADIANS :

<code>math.radians(deg)</code>	Degrés → radians
<code>math.radians(180)</code>	→ 3.141592653589793 (= π)
<code>math.radians(90)</code>	→ 1.5707963267948966 (= $\pi/2$)
<code>math.radians(360)</code>	→ 6.283185307179586 (= 2π)
<code>math.degrees(rad)</code>	Radians → degrés
<code>math.degrees(math.pi)</code>	→ 180.0
<code>math.degrees(math.pi / 4)</code>	→ 45.0

```
math.degrees(2 * math.pi) → 360.0
```

FONCTIONS TRIGONOMÉTRIQUES DIRECTES (angle en radians) :

```
math.sin(x)          Sinus
math.sin(0)          → 0.0
math.sin(math.pi / 2) → 1.0
math.sin(math.pi)    → 1.2246467991473532e-16 (≈ 0)
math.cos(x)          Cosinus
math.cos(0)          → 1.0
math.cos(math.pi)    → -1.0
math.cos(math.pi / 2) → 6.123233995736766e-17 (≈ 0)
math.tan(x)          Tangente
math.tan(0)          → 0.0
math.tan(math.pi / 4) → 0.9999999999999999 (≈ 1)
math.tan(math.pi / 2) → 1.633123935319537e+16 (≈ ∞)
```

FONCTIONS TRIGONOMÉTRIQUES INVERSES :

```
math.asin(x)          Arcsin, x ∈ [-1, 1] → [-π/2, π/2]
math.asin(1)          → 1.5707963267948966 (= π/2)
math.asin(0)          → 0.0
math.asin(2)          → ValueError
math.acos(x)          Arccos, x ∈ [-1, 1] → [0, π]
math.acos(1)          → 0.0
math.acos(-1)         → 3.141592653589793 (= π)
math.atan(x)          Arctan, x ∈ ℝ → (-π/2, π/2)
math.atan(1)          → 0.7853981633974483 (= π/4)
math.atan(math.inf)   → 1.5707963267948966 (= π/2)
math.atan2(y, x)      Arctan2 → (-π, π] (angle du vecteur)
math.atan2(1, 1)      → 0.785... (π/4, quadrant 1)
math.atan2(1, -1)     → 2.356... (3π/4, quadrant 2)
math.atan2(-1, -1)    → -2.356... (-3π/4, quadrant 3)
math.atan2(0, -1)     → 3.141592653589793 (= π)
math.atan2(0, 0)      → 0.0
# atan2 est préférable à atan(y/x) : gère les quadrants et la division par 0
```

FONCTIONS HYPERBOLIQUES :

```
math.sinh(x)          Sinus hyperbolique (e^x - e^-x) / 2
math.sinh(0)          → 0.0
math.sinh(1)          → 1.1752011936438014
math.cosh(x)          Cosinus hyperbolique (e^x + e^-x) / 2
math.cosh(0)          → 1.0
math.cosh(1)          → 1.5430806348152417
math.tanh(x)          Tangente hyperbolique
math.tanh(0)          → 0.0
math.tanh(1)          → 0.7615941559557649
math.tanh(math.inf)   → 1.0
math.tanh(-math.inf)  → -1.0
math.asinh(x)         Arcsinh, x ∈ ℝ
math.acosh(x)         Arccosh, x ≥ 1
math.acosh(1)         → 0.0
math.atanh(x)         Arctanh, x ∈ (-1, 1)
math.atanh(0)         → 0.0
```

IDENTITÉS UTILES :

```
# sin²(x) + cos²(x) = 1
math.sin(x)**2 + math.cos(x)**2 ≈ 1.0
# cosh²(x) - sinh²(x) = 1
math.cosh(x)**2 - math.sinh(x)**2 ≈ 1.0
# tanh(x) = sinh(x) / cosh(x)
```

EXEMPLES :

```
# Distance entre deux points GPS (formule haversine simplifiée)
def haversine(lat1, lon1, lat2, lon2):
    R = 6371 # km
    dlat = math.radians(lat2 - lat1)
    dlon = math.radians(lon2 - lon1)
    a = math.sin(dlat/2)**2 + math.cos(math.radians(lat1)) * \
        math.cos(math.radians(lat2)) * math.sin(dlon/2)**2
    return 2 * R * math.asin(math.sqrt(a))
# Angle d'un vecteur 2D
angle_deg = math.degrees(math.atan2(y, x))
```

DESCRIPTION : Fonctions d'erreur, gamma, et autres fonctions mathématiques avancées.
 POURQUOI : Statistiques, combinatoires, physique théorique.
 CONTEXTE : Probabilités, distributions, intégrales spéciales.

FONCTIONS D'ERREUR :

<code>math.erf(x)</code>	Fonction d'erreur <code>erf(x)</code>
<code>math.erf(0)</code>	→ 0.0
<code>math.erf(1)</code>	→ 0.8427007929497149
<code>math.erf(math.inf)</code>	→ 1.0
<code>math.erfc(x)</code>	Complémentaire <code>erfc(x) = 1 - erf(x)</code>
<code>math.erfc(0)</code>	→ 1.0
<code>math.erfc(1)</code>	→ 0.15729920705028513
<code>math.erfc(math.inf)</code>	→ 0.0
# erf et erfc sont complémentaires : <code>erf(x) + erfc(x) = 1</code>	
# CDF normale : $\Phi(x) = (1 + \text{erf}(x / \sqrt{2})) / 2$	
<code>def norm_cdf(x): return (1 + math.erf(x / math.sqrt(2))) / 2</code>	

FONCTION GAMMA :

<code>math.gamma(x)</code>	$\Gamma(x)$ – généralisée factorielle
<code>math.gamma(1)</code>	→ 1.0
<code>math.gamma(2)</code>	→ 1.0 ($\Gamma(2) = 1! = 1$)
<code>math.gamma(5)</code>	→ 24.0 ($\Gamma(5) = 4! = 24$)
<code>math.gamma(0.5)</code>	→ 1.7724538509055159 (= $\sqrt{\pi}$)
<code>math.gamma(n+1)</code>	= n! pour entiers ≥ 0
<code>math.lgamma(x)</code>	$\ln(\Gamma(x))$ – évite l'overflow
<code>math.lgamma(1000)</code>	→ 5905.220423209... ($\Gamma(1000)$ énorme)

FONCTIONS DIVERSES :

<code>math.hypot(x, y)</code>	$\sqrt{x^2 + y^2}$ – hypoténuse
<code>math.hypot(3, 4)</code>	→ 5.0
<code>math.hypot(x, y, z)</code>	$\sqrt{x^2 + y^2 + z^2}$ (Python 3.8+)
<code>math.hypot(*coords)</code>	Norme euclidienne N-D
<code>math.hypot(1, 1, 1)</code>	→ 1.7320508075688772 (= $\sqrt{3}$)
<code>math.dist(p, q)</code>	Distance euclidienne entre points
<code>math.dist([1,2,3], [4,5,6])</code>	→ 5.196152422706632 (= $\sqrt{27}$)
<code>math.dist((0,0), (3,4))</code>	→ 5.0
<code>math.prod(iterable)</code>	Produit des éléments (Python 3.8+)
<code>math.prod([1, 2, 3, 4, 5])</code>	→ 120 (= 5!)
<code>math.prod([1.5, 2.5, 3.5])</code>	→ 13.125
<code>math.prod(range(1, 6))</code>	→ 120
<code>math.prod([])</code>	→ 1 (élément neutre)
<code>math.prod(iterable, start=1)</code>	Avec valeur initiale
<code>math.fsum(iterable)</code>	Somme précise (évite l'erreur flottante)
<code>math.fsum([0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1])</code>	→ 1.0 (exact !)
<code>sum([0.1]*10)</code>	→ 0.9999999999999999 (imprécis)
<code>math.fsum([1e100, 1, -1e100])</code>	→ 1.0 (exact)
<code>sum([1e100, 1, -1e100])</code>	→ 0.0 (perd le 1 !)

EXEMPLES :

```
# Probabilité cumulée normale
x = 1.96
p = (1 + math.erf(x / math.sqrt(2))) / 2 → ≈ 0.975
# Log-vraisemblance via lgamma (évite l'overflow)
log_binom = math.lgamma(n+1) - math.lgamma(k+1) - math.lgamma(n-k+1)
```

[math] Tests et propriétés des flottants

DESCRIPTION : Vérification des cas particuliers et manipulation des flottants IEEE 754.
 POURQUOI : Débogage, gestion des cas limites, calcul numérique robuste.
 CONTEXTE : Validation de données, algorithmes numériques, gestion d'erreurs.

TESTS :

<code>math.isnan(x)</code>	Vrai si x est NaN (Not a Number)
<code>math.isnan(float("nan"))</code>	→ True
<code>math.isnan(0.0)</code>	→ False
<code>math.isnan(math.nan)</code>	→ True
<code>math.isinf(x)</code>	Vrai si x est infini ($\pm\infty$)
<code>math.isinf(float("inf"))</code>	→ True
<code>math.isinf(-float("inf"))</code>	→ True
<code>math.isinf(1e308 * 10)</code>	→ True (overflow → inf)
<code>math.isinf(0.0)</code>	→ False
<code>math.isfinite(x)</code>	Vrai si x est fini (ni NaN ni inf)
<code>math.isfinite(1.0)</code>	→ True

```

math.isfinite(math.nan)           → False
math.isfinite(math.inf)          → False
math.isclose(a, b)                Vrai si  $a \approx b$  (comparaison floue)
math.isclose(0.1 + 0.2, 0.3)     → True (!)
0.1 + 0.2 == 0.3                 → False (!)
math.isclose(a, b, rel_tol=1e-9) Tolérance relative (défaut 1e-9)
math.isclose(a, b, abs_tol=0.0)  Tolérance absolue (défaut 0.0)
math.isclose(a, b, rel_tol=1e-5, abs_tol=1e-8)
# Formule :  $|a-b| \leq \max(\text{rel\_tol} * \max(|a|, |b|), \text{abs\_tol})$ 
# Attention : math.isclose(0.0, 0.0, rel_tol=1e-9) → True
# Attention : math.isclose(0.0, 1e-10, rel_tol=1e-9) → False (utiliser abs_tol)
math.isclose(0.0, 1e-10, abs_tol=1e-9) → True

```

MANIPULATION DES FLOTTANTS :

```

math.copysign(x, y)              |x| avec le signe de y
math.copysign(3, -2)            → -3.0
math.copysign(-3, 2)           → 3.0
math.copysign(1, -0.0)         → -1.0 (signe de -0)
math.copysign(1, 0.0)          → 1.0
math.fabs(x)                   |x| en float (abs() peut retourner int)
math.fabs(-3)                  → 3.0 (float)
abs(-3)                        → 3 (int)
math.fabs(-3.5)                → 3.5
math.nextafter(x, y)            Prochain flottant vers y (Python 3.9+)
math.nextafter(0.0, 1.0)       → 5e-324 (plus petit positif)
math.nextafter(1.0, 2.0)       → 1.0000000000000002
math.nextafter(1.0, 0.0)       → 0.9999999999999998
math.ulp(x)                    Unit in the Last Place (Python 3.9+)
math.ulp(1.0)                  → 2.220446049250313e-16
math.ulp(0.0)                  → 5e-324

```

CAS SPÉCIAUX (IEEE 754) :

```

float("inf")                   +∞
float("-inf")                  -∞
float("nan")                   NaN
float("inf") + 1                → inf
float("inf") - float("inf")     → nan
float("nan") == float("nan")    → False (NaN ≠ NaN !)
math.isnan(float("nan"))       → True (test correct)
1.0 / 0.0                      → ZeroDivisionError
math.log(0)                    → ValueError
math.sqrt(-1)                  → ValueError

```

EXEMPLES :

```

# Comparaison sûre de flottants
def proche(a, b, tol=1e-9):
    return math.isclose(a, b, rel_tol=tol)
# Valeur absolue signée
def signe(x): return math.copysign(1, x)
# Vérification robuste
if math.isfinite(x) and x > 0:
    result = math.log(x)

```

[math] Récapitulatif complet de toutes les fonctions

— CONSTANTES —

math.pi	math.e	math.tau	math.inf	math.nan
---------	--------	----------	----------	----------

— ARRondi ET ENTiERS —

math.floor(x)	math.ceil(x)	math.trunc(x)
math.factorial(n)	math.comb(n, k)	math.perm(n, k)
math.gcd(a, b, ...)	math.lcm(a, b, ...)	math.isqrt(n)
math.fmod(x, y)	math.remainder(x, y)	math.modf(x)
math.frexp(x)	math.ldexp(m, e)	

— PUISSANCES ET LOGARITHMES —

math.sqrt(x)	math.cbrt(x) [3.11+]	math.pow(x, y)
math.exp(x)	math.expm1(x)	math.exp2(x) [3.11+]
math.log(x)	math.log(x, base)	math.log10(x)
math.log2(x)	math.log1p(x)	

— TRIGONOMÉTRIE —

math.sin(x)	math.cos(x)	math.tan(x)
-------------	-------------	-------------

<code>math.asin(x)</code>	<code>math.acos(x)</code>	<code>math.atan(x)</code>
<code>math.atan2(y, x)</code>	<code>math.hypot(*coords)</code>	<code>math.dist(p, q)</code>
<code>math.degrees(x)</code>	<code>math.radians(x)</code>	

— HYPERBOLIQUES —

<code>math.sinh(x)</code>	<code>math.cosh(x)</code>	<code>math.tanh(x)</code>
<code>math.asinh(x)</code>	<code>math.acosh(x)</code>	<code>math.atanh(x)</code>

— FONCTIONS SPÉCIALES —

<code>math.erf(x)</code>	<code>math.erfc(x)</code>
<code>math.gamma(x)</code>	<code>math.lgamma(x)</code>
<code>math.fsum(iterable)</code>	<code>math.prod(iterable)</code>

— TESTS ET MANIPULATION FLOTTANTS —

<code>math.isnan(x)</code>	<code>math.isinf(x)</code>	<code>math.isfinite(x)</code>
<code>math.isclose(a, b)</code>	<code>math.fabs(x)</code>	<code>math.copysign(x, y)</code>
<code>math.nextafter(x, y)</code>	<code>math.ulp(x)</code>	

[cmath] Constantes complexes

DESCRIPTION : Constantes du module `cmath` (identiques à `math` mais contexte complexe).
POURQUOI : Cohérence avec les calculs complexes.
CONTEXTE : Traitement du signal, physique quantique, électronique.

CONSTANTES :

<code>cmath.pi</code>	$\pi = 3.141592653589793$
<code>cmath.e</code>	$e = 2.718281828459045$
<code>cmath.tau</code>	$\tau = 2\pi = 6.283185307179586$
<code>cmath.inf</code>	$+\infty$ réel
<code>cmath.nan</code>	NaN réel
<code>cmath.infj</code>	$+\infty$ imaginaire pur ($0+\text{infj}$)
<code>cmath.nanj</code>	NaN imaginaire pur ($0+\text{nanj}$)

[cmath] Représentation des complexes

DESCRIPTION : Représentation cartésienne et polaire des nombres complexes.
POURQUOI : Travailler avec deux représentations selon le contexte.
CONTEXTE : Électronique (impédance), signal (spectre), géométrie.

NOMBRES COMPLEXES EN PYTHON :

<code>z = 3 + 4j</code>	Notation littérale
<code>z = complex(3, 4)</code>	Constructeur
<code>z = complex(3)</code>	$\rightarrow (3+0j)$
<code>z = complex("3+4j")</code>	Depuis une chaîne
<code>z.real</code>	Partie réelle $\rightarrow 3.0$
<code>z.imag</code>	Partie imaginaire $\rightarrow 4.0$
<code>z.conjugate()</code>	Conjugué $\rightarrow (3-4j)$
<code>abs(z)</code>	Module $ z = \sqrt{(3^2+4^2)} = 5.0$
# Représentation cartésienne : $z = x + jy$	
# Représentation polaire : $z = r * e^{(j\theta)} = r * (\cos \theta + j \sin \theta)$	

CONVERSION POLAIRE / CARTÉSIENNE :

<code>cmath.phase(z)</code>	Argument (angle) $\theta \in (-\pi, \pi]$
<code>cmath.phase(1+0j)</code>	$\rightarrow 0.0$
<code>cmath.phase(0+1j)</code>	$\rightarrow 1.5707963267948966$ ($= \pi/2$)
<code>cmath.phase(-1+0j)</code>	$\rightarrow 3.141592653589793$ ($= \pi$)
<code>cmath.phase(0-1j)</code>	$\rightarrow -1.5707963267948966$ ($= -\pi/2$)
<code>cmath.phase(1+1j)</code>	$\rightarrow 0.7853981633974483$ ($= \pi/4$)
<code>abs(z)</code>	Module $r = z = \sqrt{(x^2+y^2)}$
<code>abs(3+4j)</code>	$\rightarrow 5.0$
<code>cmath.polar(z)</code>	$\rightarrow (\text{module}, \text{phase}) = (r, \theta)$
<code>cmath.polar(3+4j)</code>	$\rightarrow (5.0, 0.9272952180016122)$
<code>r, theta = cmath.polar(z)</code>	
<code>cmath.rect(r, phi)</code>	$r * e^{(j\phi)} = r*\cos(\phi) + j*r*\sin(\phi)$
<code>cmath.rect(5, cmath.phase(3+4j))</code>	$\rightarrow (3+4j)$ environ
<code>cmath.rect(1, 0)</code>	$\rightarrow (1+0j)$
<code>cmath.rect(1, cmath.pi/2)</code>	$\rightarrow (0+1j)$ environ
<code>cmath.rect(1, cmath.pi)</code>	$\rightarrow (-1+0j)$ environ
<code>cmath.rect(0, 0)</code>	$\rightarrow 0j$

EXEMPLES :

<code>z = 3 + 4j</code>	
<code>r, theta = cmath.polar(z)</code>	$r=5.0, \text{theta}=0.927$

```

z_back = cmath.rect(r, theta)          → (3+4j)
# Rotation d'un point 2D de 90°
point = 3 + 4j
rotated = point * cmath.rect(1, cmath.pi/2) → (-4+3j) environ

```

[cmath] Fonctions élémentaires complexes

DESCRIPTION : Racines, exponentielle, logarithme et puissances pour complexes.
 POURQUOI : Les fonctions réelles sont insuffisantes pour les domaines complexes.
 CONTEXTE : Signal, contrôle, physique quantique, algèbre.

RACINE ET PUISSANCE :

```

cmath.sqrt(z)          Racine carrée complexe
cmath.sqrt(-1)         → 1j
cmath.sqrt(-4)         → 2j
cmath.sqrt(0)          → 0j
cmath.sqrt(4)          → (2+0j)
cmath.sqrt(3+4j)       → (2+1j) environ
# Toutes les racines n-ièmes d'un complexe :
# z^(1/n) donne une racine, les n racines sont : r^(1/n) * e^(j*(θ+2kπ)/n)
# Puissance complexe
(1+1j) ** 2            → 2j (opérateur Python)
cmath.exp(1j * cmath.pi) → (-1+0j) – formule d'Euler !!

```

EXPONENTIELLE ET LOGARITHME :

```

cmath.exp(z)           e^z
cmath.exp(0)           → (1+0j)
cmath.exp(1j * cmath.pi / 2) → (0+1j) (e^(jπ/2) = j)
cmath.exp(1j * cmath.pi) → (-1+1.2e-16j) (≈ -1, formule Euler)
cmath.exp(1j * 2 * cmath.pi) → (1+0j) (périodicité)
cmath.log(z)           ln(z) – logarithme naturel complexe
cmath.log(-1)          → 3.141592653589793j (= jπ)
cmath.log(1j)          → 1.5707963267948966j (= jπ/2)
cmath.log(0)           → ValueError
cmath.log(z, base)     log_base(z)
cmath.log10(z)         log10(z) complexe
cmath.log(z) == math.log(abs(z)) + 1j * cmath.phase(z) # Identité fondamentale
# ln(z) = ln|z| + j*arg(z)

```

EXEMPLES :

```

# Formule d'Euler : e^(jπ) + 1 = 0
euler = cmath.exp(1j * cmath.pi) + 1
print(abs(euler) < 1e-15)          → True
# Logarithme complexe
cmath.log(complex(-1, 0))          → 3.14159...j (= jπ)
cmath.log(complex(-1, 1e-20))      → petit positive + jπ

```

[cmath] Trigonométrie complexe

DESCRIPTION : Fonctions trigonométriques et hyperboliques pour complexes.
 POURQUOI : Les relations entre trig. et hyperbolique deviennent transparentes.
 CONTEXTE : Physique mathématique, traitement du signal, théorie des réseaux.

TRIGONOMETRIE :

```

cmath.sin(z)           sin(z) complexe
cmath.sin(0)           → 0j
cmath.sin(cmath.pi/2) → (1+0j)
cmath.sin(1j)          → 1.1752011936438014j (= j*sinh(1))
cmath.cos(z)           cos(z) complexe
cmath.cos(0)           → (1+0j)
cmath.cos(1j)          → (1.5430806348152417+0j) (= cosh(1))
cmath.tan(z)           tan(z) complexe
cmath.asin(z)          arcsin complexe → valeur principale
cmath.asin(2)          → (1.5707963...-1.3169578...j)
cmath.acos(z)          arccos complexe → valeur principale
cmath.atan(z)          arctan complexe → valeur principale

```

HYPERBOLIQUES :

```

cmath.sinh(z)          sinh(z) complexe
cmath.sinh(1j * cmath.pi / 2) → (0+1j) (= j*sin(π/2) = j)
cmath.cosh(z)          cosh(z) complexe
cmath.cosh(1j * cmath.pi) → (-1+0j) (= cos(π) = -1)
cmath.tanh(z)          tanh(z) complexe

```


<code>cmath.asinh(z)</code>	<code>arcsinh</code> complexe
<code>cmath.acosh(z)</code>	<code>arccosh</code> complexe
<code>cmath.atanh(z)</code>	<code>arctanh</code> complexe

RELATIONS FONDAMENTALES trig ↔ hyperbolique :

```
# sin(jx) = j*sinh(x)
cmath.sin(1j * x) ≈ 1j * math.sinh(x)
# cos(jx) = cosh(x)
cmath.cos(1j * x) ≈ math.cosh(x)
# sinh(jx) = j*sin(x)
cmath.sinh(1j * x) ≈ 1j * math.sin(x)
# cosh(jx) = cos(x)
cmath.cosh(1j * x) ≈ math.cos(x)
# Formule d'Euler : e^(jx) = cos(x) + j*sin(x)
cmath.exp(1j * x) ≈ math.cos(x) + 1j * math.sin(x)
```

[cmath] Tests et propriétés des complexes

DESCRIPTION : Vérification des cas particuliers pour les complexes.
 POURQUOI : Robustesse numérique avec des nombres complexes.
 CONTEXTE : Calcul scientifique, traitement du signal.

TESTS :

<code>cmath.isnan(z)</code>	Vrai si <code>z.real</code> ou <code>z.imag</code> est NaN
<code>cmath.isnan(complex(float("nan"), 0))</code>	→ True
<code>cmath.isnan(complex(0, float("nan")))</code>	→ True
<code>cmath.isnan(1+2j)</code>	→ False
<code>cmath.isinf(z)</code>	Vrai si <code>z.real</code> ou <code>z.imag</code> est infini
<code>cmath.isinf(complex(float("inf"), 0))</code>	→ True
<code>cmath.isinf(complex(0, float("inf")))</code>	→ True
<code>cmath.isinf(1+2j)</code>	→ False
<code>cmath.isfinite(z)</code>	Vrai si <code>z.real</code> et <code>z.imag</code> sont finis
<code>cmath.isfinite(1+2j)</code>	→ True
<code>cmath.isfinite(complex(float("inf"), 1))</code>	→ False
<code>cmath.isclose(a, b)</code>	Vrai si <code>a ≈ b</code> (complexes)
<code>cmath.isclose(1+2j, 1.0000001+2j, rel_tol=1e-6)</code>	→ True
<code>cmath.isclose(a, b, rel_tol=1e-9)</code>	Tolérance relative (défaut)
<code>cmath.isclose(a, b, abs_tol=0.0)</code>	Tolérance absolue

— RÉCAPITULATIF COMPLET DE CMATH —

Constantes

<code>cmath.pi</code>	<code>cmath.e</code>	<code>cmath.tau</code>	<code>cmath.inf</code>	<code>cmath.nan</code>
<code>cmath.infj</code>	<code>cmath.nanj</code>			

Représentation polaire

<code>cmath.phase(z)</code>	<code>cmath.polar(z)</code>	<code>cmath.rect(r, phi)</code>
-----------------------------	-----------------------------	---------------------------------

Puissances et logarithmes

<code>cmath.sqrt(z)</code>	<code>cmath.exp(z)</code>	<code>cmath.log(z)</code>
<code>cmath.log(z, base)</code>	<code>cmath.log10(z)</code>	

Trigonométrie

<code>cmath.sin(z)</code>	<code>cmath.cos(z)</code>	<code>cmath.tan(z)</code>
<code>cmath.asin(z)</code>	<code>cmath.acos(z)</code>	<code>cmath.atan(z)</code>

Hyperboliques

<code>cmath.sinh(z)</code>	<code>cmath.cosh(z)</code>	<code>cmath.tanh(z)</code>
<code>cmath.asinh(z)</code>	<code>cmath.acosh(z)</code>	<code>cmath.atanh(z)</code>

Tests

<code>cmath.isnan(z)</code>	<code>cmath.isinf(z)</code>	<code>cmath.isfinite(z)</code>
<code>cmath.isclose(a, b)</code>		

=====

COMPARATIF math vs cmath vs numpy

=====

FONCTION	math (réel)	cmath (complexe)	numpy (tableau)
Racine	<code>math.sqrt(x)</code>	<code>cmath.sqrt(z)</code>	<code>np.sqrt(arr)</code>
Exponentielle	<code>math.exp(x)</code>	<code>cmath.exp(z)</code>	<code>np.exp(arr)</code>
Logarithme	<code>math.log(x)</code>	<code>cmath.log(z)</code>	<code>np.log(arr)</code>
Log base 10	<code>math.log10(x)</code>	<code>cmath.log10(z)</code>	<code>np.log10(arr)</code>
Log base 2	<code>math.log2(x)</code>	<code>— (log(z,2))</code>	<code>np.log2(arr)</code>

Sin / Cos	math.sin/cos(x)	cmath.sin/cos(z)	np.sin/cos(arr)
Atan2	math.atan2(y, x)	– (utiliser phase)	np.arctan2(y, x)
Module	abs(x)	abs(z)	np.abs(arr)
Phase	– (math.atan2)	cmath.phase(z)	np.angle(arr)
Polaire	– (hypot + atan2)	cmath.polar(z)	np.abs, np.angle
FFT	–	–	np.fft.fft(arr)
Isnan	math.isnan(x)	cmath.isnan(z)	np.isnan(arr)
Isinf	math.isinf(x)	cmath.isinf(z)	np.isinf(arr)
Isclose	math.isclose(a, b)	cmath.isclose(a, b)	np.isclose(a, b)
Arrondi bas	math.floor(x)	– (x.real, x.imag)	np.floor(arr)
Arrondi haut	math.ceil(x)	–	np.ceil(arr)
Entier sqrt	math.isqrt(n)	–	–
Factorielle	math.factorial(n)	–	– (scipy.special.factorial)
Gamma	math.gamma(x)	–	scipy.special.gamma(arr)
Erf	math.erf(x)	–	scipy.special.erf(arr)
Somme précise	math.fsum(iter)	–	–
Produit	math.prod(iter)	–	np.prod(arr)
Distance	math.dist(p, q)	–	np.linalg.norm(p-q)
Hypot	math.hypot(*args)	–	np.hypot(x, y)

— QUAND UTILISER QUOI —

math → Calcul scalaire sur réels, sans numpy. Léger, rapide, standard.
 cmath → Calcul scalaire sur complexes. Indispensable pour physique, signal.
 numpy → Calculs vectorisés sur tableaux. ML, data science, traitement de signal.
 scipy → Fonctions avancées (gamma, erf, Bessel...) sur tableaux.

— PIÈGES COURANTS —

math.sqrt(-1) → ValueError, pas une solution silencieuse
 0.1 + 0.2 == 0.3 → False ! Utiliser math.isclose()
 sum([0.1]*10) == 1.0 → False ! Utiliser math.fsum()
 int(-3.7) → -3 (troncature), math.floor(-3.7) → -4 (vers -∞)
 math.log(0) → ValueError, pas -inf
 float("nan") == float("nan") → False ! Utiliser math.isnan()
 math.factorial(n) → entier Python arbitrairement grand (pas d'overflow)
 math.gcd(-12, 8) → 4 (accepte les négatifs, résultat toujours positif)
 math.pow(x, y) → float toujours, x**y peut retourner int
 cmath.sqrt(4) → (2+0j), pas 2 (toujours complexe)
 cmath.log(z) → valeur principale, partie imag ∈ (-π, π]
 math.atan2(y, x) → toujours préférer à math.atan(y/x) (gère x=0)

=====